Class-Aware Augmentation Selection for Multi-Class Object Detection

***Index Terms*—augmentation policy, data augmentation, gradient alignment, infrastructure inspection, multi-class images, object detection**

I. Introduction

Object detection underlies applications ranging from everyday scene understanding to the automated inspection of roads and bridges [1]–[3], and in most of them a detector must locate and classify objects of several classes within the same image. Achieving reliable detection remains challenging, because the appearance of each class varies with illumination, viewpoint, background and image quality, and a training set rarely covers this variation in full. The difficulty is compounded by differences among the classes themselves. Some are identified mainly by shape or fine structure, such as thin cracks in pavement and concrete [4], [5], whereas others are identified mainly by colour or texture, such as rust on steel [5], and everyday scenes routinely place small, thin objects beside large, compact ones [1]. Consequently, improving detection requires exposing the detector to a wider range of visual conditions while preserving the cues that distinguish each class.

Data augmentation addresses the first part of this requirement by applying controlled edits, such as geometric, colour and image-quality transformations, to training images, so that the detector learns from more varied conditions without additional annotation. The effectiveness of an edit, however, depends on whether it preserves the cues that define each class. Colour jitter alters the property that identifies a colour-defined class such as rust while leaving the outline of a shape-defined object intact, whereas Gaussian blur can erase a thin structure such as a hairline crack while barely changing a large, compact object. The same edit can therefore provide useful variation for one class while removing essential information from another. In detection, this tension arises within a single training image rather than only between images: standard pipelines edit the whole image, so every object in it receives the same edit, and an edit that helps the detector learn one class can hinder another class in the same image. Such conflicts are easily overlooked, because augmentation is usually judged by accuracy averaged over classes, and gains in average accuracy can coexist with losses for individual classes [6]. This motivates augmentation that accounts for how each class responds to each edit, and that chooses the edit for an image according to the classes the image contains.

Existing augmentation methods have progressively moved toward this goal, but each stops short of it. AutoAugment and RandAugment search for or simplify one policy shared by the whole dataset [7], [8], and detection-specific extensions add edits restricted to annotated boxes or adapt the policy to object scale [9], [10]; none of these policies, however, depends on the class of the objects being edited. Class-aware approaches recognize that classes may require different augmentations [11]–[13], in some cases by setting augmentation probabilities per class before training [14]. Because these policies are fixed before or apart from detector training, they cannot follow changes in what each class needs as learning progresses. Online methods remove this restriction by adapting augmentation from the feedback of the evolving model, but the feedback they use introduces two further limitations. Tolerance-based controllers such as CTAugment and CUDA adjust augmentation according to whether the model still recognizes augmented examples [15], [16], which indicates what the model can already handle rather than whether training on the edit improves it. Benefit-based methods such as BDA, DDAS and MADAug measure that improvement through a held-out loss [17]–[19], but they compute it over all classes together, so an edit that helps frequent classes can be preferred even when it harms a rare one. Moreover, the class-aware methods among them are formulated for classification, where each training sample carries one label, and therefore do not specify which class should govern an image that contains several.

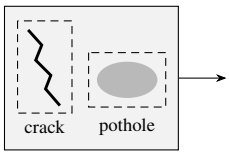
This gap matters in detection because the classes in one image can disagree. A detector may learn well from blurred images of large, compact objects, such as cars or potholes, while thin objects in the same images, such as bicycles or hairline cracks, lose the detail that identifies them. A policy guided by pooled feedback sees only the net effect of blur and may apply it to every image, whereas a policy keyed to a single class must ignore the other. What is needed instead is a measure of how much each edit helps each class learn, combined into one decision for the image that respects every class it contains.

To meet this need, we propose Class-Utility Augmentation (CUA), an online augmentation policy that selects the edit for each training image from the classes present in it. CUA is built around the *class utility* of an edit, defined as the estimated amount that the edit helps a given class learn. CUA estimates this utility during training with a first-order argument from training-influence analysis [20]: if a gradient step on the edited image would reduce a class's loss on a held-out *reference set* more than a step on the original image would, the edit currently benefits that class. For each training image, CUA

then averages the utilities of the classes present, removes any edit that is reliably harmful to one of them, and samples a single edit from the remainder, with "no edit" always available as a candidate of zero utility (Fig. 1). Because the utilities are re-estimated from the current detector, the preferred edits can change as training proceeds, and because CUA requires no policy network and no separate search phase, it adds only a periodic gradient computation to standard training.

The value of CUA rests on the premise that classes respond differently to the same edit, so we examine this premise before evaluating the method. Across road damage (RDD2022), bridge damage (dacl10k) and everyday objects (PASCAL VOC) [1], and with two detector families, Faster R-CNN [21] and RetinaNet [22], we train detectors with one edit at a time to measure its effect on each class, and with an edit applied only to the objects of one class to separate its direct effect on that class from its effect on the others. We then compare CUA with shared-utility augmentation, an otherwise identical policy that averages utility over all classes, so that any difference between the two reflects class information alone. **[One-sentence summary of findings: which edits affect classes in opposite directions; CUA minus shared-utility on test macro AP50 with 95% intervals; rank agreement between estimated utilities and measured effects.]** In summary, this paper makes the following contributions:

1) Class-Utility Augmentation, an online augmentation policy for multi-class object detection that estimates how much each edit helps each class learn and selects one edit per image from the classes it contains, with a no-edit option and a per-class harm block.
2) A controlled measurement of how individual edits affect each class in detection, across three domains and two detector families, which also separates the effect of an edit on the class it is applied to from its effect on the other classes in the image.
3) A comparison with a shared-utility counterpart that differs from CUA only in class information, together with standard augmentation baselines, and a test of whether the estimated class utilities agree with the measured per-class effects.



training image

Edit	Crack	Pothole	Policy
no edit	0	0	kept
blur	−	+	blocked (hurts crack)
saturation	+	+	most likely
translation	+	0	kept

class utility of each edit, re-estimated during training

Fig. 1. The idea behind CUA (illustrative signs, not measured values). The image contains a crack and a pothole. For each candidate edit, CUA keeps an estimate of how much the edit currently helps each class learn: its class utility (+ helps, − hurts, 0 no effect). Blur is removed because it reliably hurts the crack, although it helps the pothole. The remaining edits, including no edit, are sampled according to their utility averaged over the two classes, so saturation, which helps both, is the most likely choice. A policy shared across classes sees only the average over all classes and can still apply blur.

II. Related Work

Augmentation policies for object detection. Automatic augmentation began by treating the choice of edits as a search over a whole dataset. AutoAugment searches over operations, probabilities and magnitudes with validation accuracy as the reward [7]; because this search is expensive, RandAugment reduces it to two shared parameters [8], and TrivialAugment goes further, showing that sampling one edit and strength uniformly at random is a strong baseline that needs no tuning [23]. Detection raises an additional question, namely where in the image an edit should act. Zoph *et al.* answer it by learning policies that include edits restricted to the content of each box [9], Scale-Aware AutoAug by conditioning image- and box-level policies on object scale [10], and InterAug by constructing a tuning-free detection policy [24]. Inspection work has followed the same pattern with recipes designed for damage imagery, such as bounding-box augmentation combined with samples from generative adversarial networks [25] or content- and perspective-aware cut-and-paste of damage [26]. Although these methods differ in how the policy is found, they share one property: the edit for an image is drawn from a single distribution, conditioned at most on location or scale, which implicitly assumes that every class benefits from the same edits.

Class-aware augmentation. Evidence from recognition suggests that this assumption does not always hold. Kirichenko *et al.* show that augmentation which raises average ImageNet accuracy can lower the accuracy of individual classes [6], and Tsuchiya *et al.* find that augmentation methods for road-damage classification differ in their effect on individual damage classes [11]. Several methods have therefore made augmentation depend on the class. Yoo and Kang derive class-specific parameters from a classifier trained beforehand [12], AdaAug learns class-adaptive policies in a separate search [13], CUDA updates a severity for each class during training [16], and Pham *et al.* set augmentation probabilities for each damage type from an exploration of the data [14]. These methods share a limitation that becomes decisive in detection. The classification methods select the augmentation by the label of a sample, which has no answer when an image contains objects of several classes, while the detection recipe sidesteps the question by fixing its probabilities before training. PSIS, the closest detection counterpart, adapts to per-class detection accuracy during training, but it controls how many instance-switched samples each class receives rather than which edit each image receives [27]. A method for multi-class detection must therefore choose edits from the whole set of classes in an image, and must do so as the needs of the detector change during training.

Adapting augmentation during training. Online methods make this second requirement possible by updating augmentation from the model as it trains, but they differ in the feedback they treat as evidence that an edit is useful. Tolerance-based methods ask whether the current model still recognizes,

or tolerates, augmented examples. CTAugment adjusts per-operation magnitudes from predictions on augmented labeled data, pooled over classes [15]; CUDA raises or lowers the severity of each class according to how well its augmented examples are recognized [16]; and TeachAugment keeps learned transformations recognizable to a teacher while making them hard for the target model [28]. Recognition, however, reflects what the model can already handle rather than what it gains from training on the edit, which is the quantity augmentation is meant to increase. Benefit-based methods measure this gain more directly. BDA rewards an augmentation by the training gain it yields over a paired unaugmented reference [17], DDAS uses a one-step gradient update to estimate how the augmentation distribution affects a held-out loss, including for detection [18], MADAug learns an input-dependent policy through a bilevel validation objective [19], and RangeAugment and AdaAugment adapt magnitudes from task-loss feedback [29], [30]. Their common limitation is that the benefit is computed over all classes together, so a gain for frequent classes can outweigh a loss for a rare one, which is precisely the trade-off that a multi-class image creates. Estimating the gain separately for each class need not be costly. TracIn measures the influence of a training example on a test loss through the alignment of their gradients [20], and CUA applies the same first-order argument to edited versus unedited images and to a separate reference loss for each class.

Table I places these methods on two axes: the signal that guides augmentation, and whether that signal is resolved by class. Each ingredient of CUA has a precedent among them, including class conditioning, online adaptation, benefit signals and gradient-alignment estimates. None of them, however, keeps a benefit estimate separate for each class and combines these estimates into one decision for an image that contains several classes, with leaving the image unedited as an explicit option. CUA is designed for this case, and its comparison with shared-utility augmentation, which removes the class information and nothing else, tests whether the class information matters.

III. Methodology

A. Problem Formulation and Overview

We consider road-damage detection from complete images annotated with bounding boxes and damage categories. Let $\mathcal{D} = \{(x_i, Y_i)\}_{i=1}^{N}$ denote the training set, where $Y_i = \{(b_{ij}, c_{ij})\}_{j=1}^{n_i}$ contains the boxes and class labels in image x_i. The target categories are longitudinal cracks, transverse cracks, alligator cracks, and potholes, following RDD2022 [4]. A detector f_θ maps each image to predicted damage locations and categories. Within this task, our objective is to select the transformation family and strength for a local damage region according to its class composition and the detector's current response.

Local augmentation requires a consistent decision wherever annotated boxes share pixels. We therefore organize connected overlapping boxes into groups and assign one transformation to each selected group. The framework regulates transformation strength through class-specific detection retention and selects transformation families using class-balanced loss feedback. Separate groups receive policies conditioned on their own classes, while all classes within an overlap group constrain its shared edit. Ground-truth annotations guide this process during training; inference uses the detector alone.

B. Overall Framework

The controller maintains an augmentation policy for each class set represented in an overlap group. Denoting this set by s, a group containing a single damage category uses $s = \{c\}$, whereas a mixed-class group uses its complete set of categories. For each s, the policy stores a strength for each transformation family and a probability of selecting that family. Groups with the same class set share a policy, and their probe measurements are pooled separately for each represented class. With four categories, this representation contains at most 15 class sets and requires no additional policy network.

Figure 2 summarizes the interaction between the controller and detector. After an initial warm-up, the controller evaluates paired reference and transformed images using a fixed detector state. These probes determine the strongest eligible level of each family from class-specific retention, then assign family-selection probabilities from normalized loss increases. The resulting policies generate local transformations for detector training. Repeating this update at fixed intervals connects the augmentation choices to the detector's changing response throughout optimization.

C. Overlap Groups and Local Transformations

Overlap groups define the spatial units to which the controller assigns transformations. We first clip boxes to the image boundary and rasterize them using the same pixel-support convention as the edit masks. A graph then connects boxes whose rasterized regions have positive-area intersection. Its connected components form the groups, with an isolated box forming a group of its own. Grouping is based on the annotated box regions, including the pavement context they contain.

For a selected group G, the transformation acts on the union M_G of its box regions. Let $T_{o,m}$ denote operation o at discrete strength m. The augmented image is

$$\widetilde{x} = x + \alpha_G \odot \big(T_{o,m}(x) - x\big), \tag{1}$$

where $\odot$ denotes pixelwise multiplication and α_G is a blending mask with values in $[0, 1]$. The mask is zero outside M_G and tapers inward at the union boundary to blend the transformed content with its surroundings. We compute $T_{o,m}(x)$ from the reference image and composite it once over the union. This construction gives shared pixels a single transformation and makes the result independent of box-processing order.

The transformation bank contains two coordinate-preserving families: photometric transformations, comprising brightness, contrast, and gamma adjustment; and blur/noise transformations, comprising Gaussian blur and additive Gaussian noise. Opposite directions, such as brightening and darkening, are

TABLE I
AUGMENTATION METHODS GROUPED BY FEEDBACK SIGNAL AND CLASS RESOLUTION

Method	Original setting	Signal	Per class	Relation to CUA
TrivialAugment [23]	Classification	None	No	Uniform augmentation baseline in our comparison.
Zoph *et al.* [9]	Detection	Offline search	No	Box-level operations; one policy for the dataset.
Scale-Aware AutoAug [10]	Detection	Offline search	No	Conditioned on object scale rather than class.
Pham *et al.* [14]	Road-damage detection	Data exploration	Yes	Per-class probabilities fixed before training.
PSIS [27]	Detection	Validation AP	Yes	Adapts per-class instance switching, not the edit.
Yoo and Kang [12]	Classification	Offline, then schedule	Yes	One label per sample selects the parameters.
AdaAug [13]	Classification	Learned offline	Yes	One label per sample; policy reused after search.
CTAugment [15]	Semi-supervised classification	Tolerance	No	Statistics pooled across classes.
CUDA [16]	Long-tailed classification	Tolerance	Yes	One label per sample selects the severity.
BDA [17]	Classification	Benefit	No	Paired training gain, pooled across classes.
DDAS [18]	Classification, detection	Benefit	No	One-step held-out loss, pooled across classes.
MADAug [19]	Classification	Benefit (learned)	Implicit	Input-dependent policy learned from one pooled validation loss.
RangeAugment [29]	Classification, detection	Task loss (learned)	No	Learns magnitude ranges online.
AdaAugment [30]	Classification	Loss feedback (learned)	No	Instance-level magnitudes from an actor–critic controller.
CUA (ours)	Multi-class detection	Benefit	Yes	Per-class benefit combined over the classes in each image; no-edit option and per-class harm block.

Signal: *tolerance* = whether the current model still recognizes augmented examples; *benefit* = whether training on augmented data reduces a held-out or reference loss. CTAugment is the augmentation component of ReMixMatch.

represented as separate operations. Each operation has a fixed mapping from ordered levels $m \in \{1, \ldots, M\}$ to its physical parameters, with level zero denoting identity. The same parameter mappings, noise distribution, intensity-clipping rule, and boundary taper are used during probing and training. Because these operations preserve spatial coordinates, the original bounding boxes remain the detection targets.

D. Paired Detection Feedback

Paired probes measure the detector's response to each candidate transformation under a common model state. At each policy update, we select a panel of target-bearing training images, preferring distinct source-image clusters, and sample one group uniformly within each image. Each source-image cluster comprises an image and any linked exact or perceptual duplicate candidates; it is distinct from the within-image overlap group. Panel selection is independent of detection outcomes, and measurements are pooled by the selected groups' class sets. The same panel evaluates the operation–strength grid, with only the target group transformed. Probes run in evaluation mode without gradient updates, keeping detector parameters, normalization statistics, preprocessing, and post-processing fixed. For noise, a fixed set of seeded draws is reused across strength levels.

Detection retention measures the fraction of reference-detected objects that remain detected after transformation. Predictions are matched one-to-one to annotated boxes using the correct class label and fixed confidence and intersection-over-union thresholds. Let d_j^0 indicate whether reference object j is detected, and let $d_j^{o,m}$ denote its transformed detection indicator, averaged over draws for a stochastic operation. For class-c objects $\mathcal{P}_{s,c}$ in the target groups of class set s, retention is

$$R_{c|s,o,m} = \frac{\sum_{j \in \mathcal{P}_{s,c}} d_j^0 d_j^{o,m}}{\sum_{j \in \mathcal{P}_{s,c}} d_j^0}. \tag{2}$$

Conditioning on reference detections separates missed detections introduced by the edit from those already present in the reference view.

The controller uses a class-set policy only when every represented class has the required minimum numbers of reference-detected objects and contributing source-image clusters within that exact class set. Repeated noise draws do not increase either count. Measurements from other class sets cannot supply missing support. An absent or insufficiently supported class set receives identity, and its policy is recomputed from the next panel rather than carried forward from an earlier update. Per-object detection indicators, matched-object IoUs, and full-image false-positive counts are retained alongside the target-

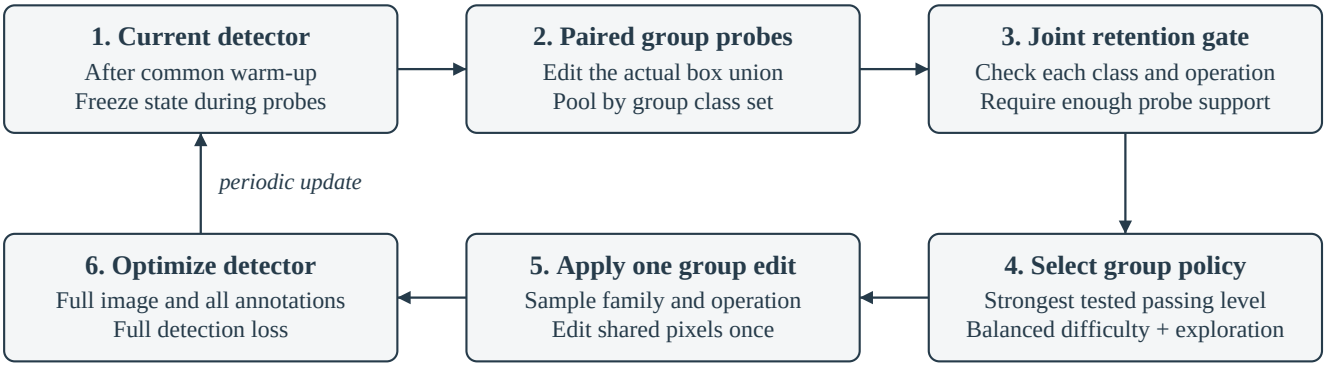


Fig. 2. Class-adaptive augmentation framework. Paired probes measure the response to a local union edit and pool feedback by group class set. Class-specific retention determines eligible family strengths, and class-balanced loss feedback determines family-selection probabilities. The selected transformation is applied once to a sampled group before full-image detector optimization.

group measurements to characterize responses beyond the edited region.

E. Joint Strength Selection

Strength selection uses a joint retention constraint across the classes sharing a group. Let $\mathcal{O}_f$ denote the operations in family f, including their directions, and let $\mathcal{A}_{s,f}$ be the set of tested nonzero levels with sufficient probe support for which

$$\min_{c \in s} \min_{o \in \mathcal{O}_f} R_{c|s,o,m} \geq \rho, \tag{3}$$

where ρ is the retention threshold. The controller selects the strongest level in this admissible set:

$$m^*_{s,f} = \max\big(\mathcal{A}_{s,f} \cup \{0\}\big). \tag{4}$$

The zero value marks a family as unavailable when no tested level satisfies the constraint. Each level is evaluated independently; selecting the strongest admissible level does not assume that retention decreases monotonically with strength. Both eligibility and strength are recomputed from the current probe measurements at each policy update.

The minimum in Eq. 3 gives every represented class and operation a separate constraint on the shared strength. For example, a strong blur level is rejected for crack–pothole groups when crack retention falls below ρ, even if pothole retention remains high. A weaker blur/noise level is eligible only when both blur and noise satisfy the threshold for both classes. Applying the constraint before family selection prevents an acceptable average from masking a failed class-level check and permits each family to retain its own admissible strength.

F. Class-Balanced Family Selection

Family selection uses an auxiliary object-level loss to measure difficulty independently of thresholded detection success. For Faster R-CNN, we assign positive region proposals to annotated objects using the detector's region-of-interest (ROI) matcher and append one ground-truth ROI explicitly assigned to each object. This additional ROI supplies a measurement even when the proposal network misses an object. Each ROI contributes classification cross-entropy plus class-specific Smooth L1 box-regression loss, with equal weights and a Smooth L1 transition parameter of $1/9$; the regression term is summed over the four box coordinates. We average ROI contributions within each object before aggregating objects.

Ground-truth ROIs are used only by this auxiliary observer and never enter the predictions used for retention or evaluation. Both views use the same assignment and loss procedure, although their proposal sets may differ. Only losses attributable to target-group objects guide family selection. Background classification and proposal-network losses remain part of ordinary detector optimization. The auxiliary score therefore measures a specific supervised response to the edit, rather than the detector's complete training objective or a guarantee of downstream benefit.

For each class, the difficulty score is the normalized increase in its mean object loss. Let $\bar{\ell}^0_{c|s}$ and $\bar{\ell}^{o,m}_{c|s}$ denote mean losses over the same class-c objects in the reference and transformed views, respectively, with stochastic draws also averaged. We define

$$A_{c|s,o,m} = \mathrm{clip}_{[0,B]}\left(\frac{\bar{\ell}^{o,m}_{c|s} - \bar{\ell}^0_{c|s}}{\bar{\ell}^0_{c|s} + \delta}\right), \tag{5}$$

where $\delta > 0$ stabilizes normalization and $B > 0$ caps the score. Averaging before clipping retains both positive and negative object-level changes when measuring the net class response. Reference-loss normalization adjusts for differences in class loss scale. The score for family f at its selected strength is

$$D_{s,f} = \frac{1}{|s|\,|\mathcal{O}_f|} \sum_{c \in s} \sum_{o \in \mathcal{O}_f} A_{c|s,o,m^*_{s,f}}. \tag{6}$$

The two averages give equal weight to each represented class and to each operation within the family, so classes with more boxes do not dominate family allocation.

The sampling distribution combines difficulty weighting with uniform exploration over eligible families. For nonempty

$\mathcal{E}_s = \{f : m^*_{s,f} > 0\}$,

$$p(f \mid s) = (1-\varepsilon)\frac{\exp(D_{s,f}/\tau)}{\sum_{h\in\mathcal{E}_s}\exp(D_{s,h}/\tau)} + \frac{\varepsilon}{|\mathcal{E}_s|}, \quad (7)$$

where $\tau > 0$ controls the concentration of the difficulty-weighted component and $0 < \varepsilon \leq 1$ sets the uniform exploration weight. The uniform term gives every eligible family a nonzero probability. After selecting a family, we sample one of its operations uniformly and apply it at $m^*_{s,f}$. Stochastic operations use a fresh draw from the same distribution used in probing. If no family is eligible, the group receives the identity transformation.

G. Training Procedure and Evaluation Design

Local augmentation is integrated into training through a fixed application probability q. After warm-up, an image selected for augmentation contributes one uniformly sampled group, whose class-set policy determines the family, operation, and strength. Other groups remain unchanged on that visit and can be selected on subsequent visits. This matches the single-group intervention used in probing. The complete image and all its annotations contribute to the detector's classification, localization, and background losses. Images without target boxes receive no local edit and retain background supervision. Policy updates use forward measurements, while detector updates minimize the full detection loss.

Policy adaptation uses only training annotations and training-image probes. Validation selects the detector checkpoint and supplies development comparisons; it does not supply the online retention or difficulty measurements. Test data are excluded from optimization, probing, and checkpoint selection. Separate random-number streams govern image order, augmentation decisions, and probe selection. Saving the controller state with the detector, optimizer, and random-number states preserves the adopted policy across interrupted training.

The evaluation contrasts identity augmentation, fixed image-level augmentation, fixed local augmentation, online shared local control, and online class-adaptive local control. Section IV specifies the common detector protocol and the comparisons' scope. Detection retention constrains the measured response of the current model; neither passing this constraint nor increasing the auxiliary loss establishes semantic preservation or improved generalization.

IV. Experimental Setup

A. Dataset and Partitioning

We use complete annotated images from the seven RDD2022 source groups: China Drone, China MotorBike, Czech, India, Japan, Norway, and the United States [4]. The four foreground labels are D00, D10, D20, and D40. Auditing the 38,385 available images excludes one exact duplicate with conflicting annotations and one image with an ambiguous class label; one zero-area target box is removed while retaining its image and remaining targets. The resulting pool contains 38,383 images and 55,002 target objects. Valid difficult and truncated targets are retained as ordinary ground truth. Images without annotations for the four target classes remain in the dataset as background-supervision examples; this designation does not establish that their pavement is damage-free.

The study uses a custom mixed-source split of 26,863 training, 5,760 validation, and 5,760 reserved test images, containing 38,530, 8,225, and 8,247 target objects, respectively. Exact matches and screened perceptual-similarity links define source-image components that are assigned wholly to one partition. Existing test assignments are preserved, components with prior training exposure remain in training, and unassigned groups are balanced within each source toward a 70/15/15 allocation. Six validation images linked to components with training exposure are reassigned to training through this rule. The manifests and exclusions are fixed before detector training. This grouping reduces leakage through identified related images but does not establish independence of all roads or capture sequences, and the split does not measure unseen-country transfer.

B. Detector and Optimization

All conditions use Faster R-CNN with a ResNet-50 feature pyramid backbone, initialized from the same COCO-pretrained weights. The ROI predictor is replaced with a five-class head for the four damage categories and background. All backbone stages are trainable, with frozen batch-normalization statistics. Each run starts afresh from this initialization rather than continuing a previous condition's checkpoint. Images are decoded in stored orientation, converted to RGB, resized with aspect ratio preserved, and centered in a 640×640 canvas with padding value 114. Bounding boxes follow the actual horizontal and vertical resize factors; predictions are mapped back to original image coordinates for evaluation.

Training uses 20 epochs, a batch size of four, and stochastic gradient descent with momentum 0.9 and weight decay 5×10^{-4}. The base learning rate is 0.0025, reached through a linear warm-up over 1,000 optimizer steps from 0.001 times that value. It decreases by a factor of ten after epochs 12 and 16. The complete budget is 134,320 optimizer steps and 537,260 training-image presentations per condition. Augmented conditions use an unaugmented first epoch before enabling their policies. Computation uses FP32 with automatic mixed precision and TF32 disabled. The implementation uses PyTorch 2.3.0, TorchVision 0.18.0, and one NVIDIA A10 GPU per run on CCAST.

C. Augmentation Conditions

Table II defines the five comparison conditions. Identity applies only the common preprocessing. The fixed image-level and fixed local conditions share the same operation parameters and sampling rule: each family has probability $1/2$, followed by a uniform draw among its operations at level two. Their difference is whether the edit covers the resized image content or one sampled overlap-group union. Across all augmented conditions, only target-bearing images are eligible, with application probability $q = 0.5$ after the first

epoch; images without target boxes remain unchanged. Thus, the spatial comparison does not also introduce augmentation of background-only images.

The two online conditions use identical probe panels, group choices, candidate transformations, update schedules, and numerical thresholds. The shared controller pools groups across class sets and applies its retention constraint to all four classes. The class-adaptive controller pools only groups with the same exact class set and constrains each class represented in that set. This contrast changes feedback pooling and policy routing, including the support available to each policy. The fixed-local comparison additionally changes access from one prescribed strength to three candidate strengths, so it cannot isolate online adaptation independently of the candidate range. The fixed recipes are specified before their runs and are not selected by a validation policy search.

D. Transformations and Controller Settings

The operation bank contains six photometric operations and two blur/noise operations, with the three strength levels in Table III. Brightness scales RGB values; contrast scales deviations from the full-content mean luminance; gamma applies the exponent to values normalized to $[0, 1]$. Noise is independently Gaussian for each pixel and channel. Operations act on stored RGB code values, followed by clipping and rounding to 8-bit intensities. For local edits, clipped boxes are rasterized using floor at the minimum edges and ceiling at the maximum edges. The union mask has opacity 0.5 on its outer eight-neighbor boundary and 1 in its one-pixel-eroded interior; one-pixel-wide regions retain opacity 0.5. Padding remains unchanged.

Controller updates occur before epochs 2, 4, . . . , 20, with the selected policies held fixed between updates. Each update uses 256 target-bearing training images in total, not 256 per class set. Images are ranked using keyed pseudorandom draws, prioritizing distinct source-image clusters before filling from unused images, and one group is selected uniformly in each image. Gaussian noise uses two independent draws, each reused across strength levels; the other operations use one view per level. Table IV gives the remaining settings. They are fixed engineering choices for this comparison, rather than validation-optimized estimates. The minimum support prevents decisions based on only one or two detections, but does not supply a statistical precision guarantee. Unsupported or inadmissible class sets receive identity without borrowing a shared policy or resampling another group.

E. Evaluation and Computational Accounting

Evaluation uses unaugmented images and averages precision over the four foreground classes and IoU thresholds $0.50{:}0.05{:}0.95$ (AP50:95). Validation is performed after every epoch, and the checkpoint with the highest validation AP50:95 is selected, retaining the earlier epoch on a tie. The metric implementation is pycocotools 2.0.10; complementary outputs include AP50, AP75, per-class AP, and recall with at most 100 detections. Inference uses a score threshold of 0.001 and class-wise nonmaximum suppression at IoU 0.5. Predicted boxes on images without target annotations are counted separately at confidence 0.25. Validation scores are development estimates subject to checkpoint selection; the reserved test partition is excluded from this comparison.

The comparison uses one initialization and sampling seed, 42, with common data order, optimizer settings, and checkpoint-selection rules. Keyed augmentation and probe draws are separated from the detector's random-number stream, but CUDA execution is not constrained to produce identical trajectories. Consequently, small differences cannot be separated from training variability by this design, and the comparison does not estimate between-seed uncertainty. It also does not independently ablate the retention gate, family-specific strength, or difficulty-based allocation.

The optimizer budget is identical across conditions, while online control adds forward computation. Each probe image requires one reference view and 27 transformed views: seven deterministic operations at three levels and two noise draws at each of three levels. Ten updates over 256 images therefore require 71,680 additional image forwards per online condition, plus the auxiliary ROI-head evaluations. Total run time and probe time are recorded separately, alongside policy support, retention, chosen strengths, family probabilities, and actual edit exposure. Equal augmentation opportunities do not imply equal numbers of modified images, because an online policy can select identity.

REFERENCES

[1] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The Pascal visual object classes (VOC) challenge," *Int. J. Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.

[2] J. Shang, A. A. Zhang, Z. Dong, H. Zhang, and A. He, "Automated pavement detection and artificial intelligence pavement image data processing technology," *Automation in Construction*, vol. 168, p. 105797, 2024.

[3] A. Mulyanto, R. F. Sari, A. Muis, and R. Harwahyu, "Vision-based automated pavement distress inspection: A review," *IEEE Access*, vol. 13, pp. 115 474–115 502, 2025.

[4] D. Arya, H. Maeda, S. K. Ghosh, D. Toshniwal, and Y. Sekimoto, "RDD2022: A multi-national image dataset for automatic road damage detection," *Geoscience Data Journal*, vol. 11, no. 4, pp. 846–862, 2024.

[5] J. Flotzinger, P. J. Rösch, and T. Braml, "dacl10k: Benchmark for semantic bridge damage segmentation," in *Proc. IEEE/CVF Winter Conf. Applications of Computer Vision (WACV)*, 2024, pp. 8611–8620.

[6] P. Kirichenko, M. Ibrahim, R. Balestriero, D. Bouchacourt, S. R. Vedantam, H. Firooz, and A. G. Wilson, "Understanding the detrimental class-level effects of data augmentation," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://papers.neurips.cc/paper_files/paper/2023/hash/38c05a5410a6ab7eeeb26c9dbebbc41b-Abstract-Conference.html

[7] E. D. Cubuk, B. Zoph, D. Mané, V. Vasudevan, and Q. V. Le, "AutoAugment: Learning augmentation strategies from data," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 113–123. [Online]. Available: https://openaccess.thecvf.com/content_CVPR_2019/html/Cubuk_AutoAugment_Learning_Augmentation_Strategies_From_Data_CVPR_2019_paper.html

[8] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, "RandAugment: Practical automated data augmentation with a reduced search space," in *Advances in Neural Information Processing Systems*, vol. 33, 2020. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/d85b63ef0ccb114d0a3bb7b7d808028f-Abstract.html

TABLE II
AUGMENTATION CONDITIONS UNDER A COMMON DETECTOR TRAINING PROTOCOL

Condition	Spatial scope	Feedback pooling	Strength and family selection
Identity	No augmentation	None	Identity only
Fixed image-level	Full resized content	None	Level two; uniform families
Fixed local	One overlap-group union	None	Level two; uniform families
Online shared local	One overlap-group union	All probe groups, by class	Shared family strengths and probabilities
Online class-adaptive local	One overlap-group union	Exact group class set, by class	Class-set-specific strengths and probabilities

TABLE III
OPERATION PARAMETERS AT THE THREE CANDIDATE STRENGTHS

Parameter	Level 1	Level 2	Level 3
Brightness down/up	0.9/1.1	0.8/1.2	0.7/1.3
Contrast down/up	0.9/1.1	0.8/1.2	0.7/1.3
Gamma brighten/darken	0.9/1.1	0.8/1.2	0.7/1.3
Gaussian blur radius (px)	0.4	0.8	1.2
Gaussian noise std. (RGB/255)	0.01	0.02	0.03

TABLE IV
SETTINGS SHARED BY THE TWO ONLINE CONTROLLERS

Setting	Value
Unaugmented warm-up	1 epoch
Policy updates before epochs	$2, 4, \ldots, 20$
Probe panel size per update (total)	256 images
Noise draws per level	2
Reference detection confidence / IoU	0.25 / 0.50
Retention threshold ρ	0.80
Reference support per constrained class	8 objects, 4 clusters
Candidate nonzero strengths M	3
Loss stabilizer δ / cap B	0.1 / 5
Softmax temperature τ	0.25
Uniform exploration weight ε	0.20
Application probability q	0.50

[9] B. Zoph, E. D. Cubuk, G. Ghiasi, T.-Y. Lin, J. Shlens, and Q. V. Le, "Learning data augmentation strategies for object detection," in *Computer Vision – ECCV 2020*, 2020, pp. 566–583. [Online]. Available: https://arxiv.org/abs/1906.11172

[10] Y. Chen, Y. Li, T. Kong, L. Qi, R. Chu, L. Li, and J. Jia, "Scale-aware automatic augmentation for object detection," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2021. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2021/papers/Chen_Scale-Aware_Automatic_Augmentation_for_Object_Detection_CVPR_2021_paper.pdf

[11] H. Tsuchiya, S. Fukui, Y. Iwahori, Y. Hayashi, W. Achariyaviriya, and B. Kijsirikul, "A method of data augmentation for classifying road damage considering influence on classification accuracy," *Procedia Computer Science*, vol. 159, pp. 1449–1458, 2019.

[12] J. Yoo and S. Kang, "Class-adaptive data augmentation for image classification," *IEEE Access*, vol. 11, pp. 26 393–26 402, 2023.

[13] T.-H. Cheung and D.-Y. Yeung, "AdaAug: Learning class- and instance-adaptive data augmentation policies," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022. [Online]. Available: https://openreview.net/forum?id=rWXfFogxRJN

[14] V. Pham, C. Pham, and T. Dang, "Road damage detection and classification with Detectron2 and Faster R-CNN," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, 2020, pp. 5592–5601.

[15] D. Berthelot, N. Carlini, E. D. Cubuk, A. Kurakin, H. Zhang, C. Raffel, and K. Sohn, "ReMixMatch: Semi-supervised learning with distribution alignment and augmentation anchoring," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2020. [Online]. Available: https://arxiv.org/abs/1911.09785

[16] S. Ahn, J. Ko, and S.-Y. Yun, "CUDA: Curriculum of data augmentation for long-tailed recognition," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2302.05499

[17] S. Lu, M. Zhao, S. Yuan, X. Wang, L. Yang, and D. Niu, "BDA: Bandit-based transferable AutoAugment," in *Proc. SIAM Int. Conf. Data Mining (SDM)*, 2023, pp. 550–558.

[18] A. Liu, Z. Huang, Z. Huang, and N. Wang, "Direct differentiable augmentation search," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021, pp. 12 199–12 208.

[19] C. Hou, J. Zhang, and T. Zhou, "When to learn what: Model-adaptive data augmentation curriculum," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2023, pp. 1717–1728. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2023/html/Hou_When_to_Learn_What_Model-Adaptive_Data_Augmentation_Curriculum_ICCV_2023_paper.html

[20] G. Pruthi, F. Liu, S. Kale, and M. Sundararajan, "Estimating training data influence by tracing gradient descent," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[21] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2017.

[22] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2999–3007.

[23] S. G. Müller and F. Hutter, "TrivialAugment: Tuning-free yet state-of-the-art data augmentation," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021, pp. 754–762.

[24] K. Thopalli, D. S, and J. J. Thiagarajan, "InterAug: A tuning-free augmentation policy for data-efficient and robust object detection," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV) Workshops*, 2023, pp. 253–261. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2023W/VIPriors/papers/Thopalli_InterAug_A_Tuning-Free_Augmentation_Policy_for_Data-Efficient_and_Robust_Object_ICCVW_2023_paper.pdf

[25] N. Aghayan-Mashhady and A. Amirkhani, "Road damage detection with bounding box and generative adversarial networks based augmentation methods," *IET Image Processing*, vol. 18, no. 1, pp. 154–174, 2024.

[26] P. Siripathitti, F. Forest, and O. Fink, "Cut-and-paste with precision: A content and perspective-aware data augmentation for road damage detection," in *Proc. 34th European Safety and Reliability Conf. (ESREL)*, 2024. [Online]. Available: https://arxiv.org/abs/2406.18586

[27] H. Wang, Q. Wang, F. Yang, W. Zhang, and W. Zuo, "Data augmentation for object detection via progressive and selective instance-switching," arXiv:1906.00358, 2019.

[28] T. Suzuki, "TeachAugment: Data augmentation optimization using teacher knowledge," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 10 904–10 914. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2022/html/Suzuki_TeachAugment_Data_Augmentation_Optimization_Using_Teacher_Knowledge_CVPR_2022_paper.html

[29] S. Mehta, S. Naderiparizi, F. Faghri, M. Horton, L. Chen, A. Farhadi, O. Tuzel, and M. Rastegari, "RangeAugment: Efficient online augmentation with range learning," arXiv:2212.10553, 2022.

[30] S. Yang, P. Li, X. Xiong, F. Shen, and J. Zhao, "AdaAugment: A tuning-free and adaptive approach to enhance data augmentation," *IEEE Transactions on Image Processing*, vol. 34, pp. 4843–4855, 2025.